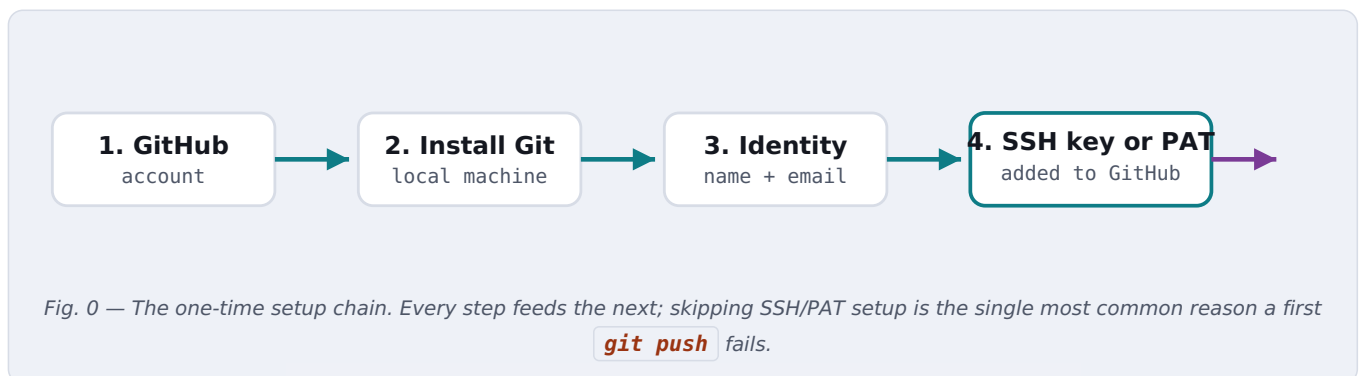


Git & GitHub · Gold Standard Reference

Prepared by: Marjan Rafi, DevOps Engineer, Pipeline Inc, Nihonbashi, Tokyo

00Getting Started — Zero to a Working Setup

Before any command in this reference is useful, Git has to actually be installed, configured, and able to talk to GitHub. This section is the part most references skip — everything between "I have never touched Git" and "my first push worked."



ONE-STOP TECH LEARNING PLATFORM

0.1 Create a GitHub Account

Go to `github.com`, sign up with a real email you'll check, and pick a username deliberately — this becomes part of your professional identity (it shows up in URLs, on your résumé, in commit history at every company you work at). Verify the email before continuing; some GitHub features are locked until you do.

0.2 Install Git

OS	Command
Windows	Download the installer from <code>git-scm.com</code> , run it with default options, then use Git Bash (installed alongside it) as your terminal.
macOS	<code>xcode-select --install</code> (ships Git via Apple's Command Line Tools) — or <code>brew install git</code> for the latest version via Homebrew.
Linux (Debian/Ubuntu)	<code>sudo apt update && sudo apt install git</code>
Linux (Fedora)	<code>sudo dnf install git</code>

```
# Confirm it worked – should print a version number
git --version
```

0.3 Set Your Identity

Git stamps every commit with a name and email — this has nothing to do with your GitHub login, and won't be checked against it unless you deliberately verify commit emails later.

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"

# Confirm:
git config --list
```

0.4 SSH Keys — Generate, Add, Verify

An SSH key is a matched pair of files: a **private key** that never leaves your machine, and a **public key** you hand to GitHub. GitHub uses the public key to verify it's really you, without you typing a password on every single push.

```
# 1. Generate a key pair (ed25519 is the modern, recommended type)
ssh-keygen -t ed25519 -C "your_email@example.com"
# Press Enter to accept the default file location.
# A passphrase is optional but recommended — you'll be prompted for it
# occasionally, not on every push, once ssh-agent is holding it (next step).

# 2. Start the SSH agent and load your new key into it
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# 3. Print your PUBLIC key so you can copy it
cat ~/.ssh/id_ed25519.pub
```

4 On GitHub: **Settings** → **SSH and GPG keys** → **New SSH key**. Paste the *public* key (the `.pub` file's content) — never the private one.

5 Verify the connection: `ssh -T git@github.com`. A successful setup replies `Hi <username>! You've successfully authenticated...`

Never paste the contents of the private key file (the one *without* `.pub`) anywhere — not into GitHub, not into a chat, not into a repo. Anyone with it can authenticate as you.

0.5 Personal Access Tokens (PAT)

GitHub removed plain password authentication for Git operations in 2021 — cloning or pushing over **HTTPS** now requires a PAT used in place of a password. (SSH, above, is a separate authentication path and doesn't need one.)

- 1 GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
- 2 Scope it to the specific repositories it needs, not "all repos" — fine-grained tokens are safer than the older "classic" tokens, which grant broad account-wide access.
- 3 Copy the token immediately — GitHub shows it exactly once and never displays it again.

When Git prompts for a password on an HTTPS clone/push, paste the PAT instead.

```
git clone https://github.com/username/repo.git
```

Username: your_username

Password: <paste your PAT here, not your GitHub password>

Avoid retyping it every time – cache it:

```
git config --global credential.helper cache # temporary, in-memory
```

or, on macOS:

```
git config --global credential.helper osxkeychain # persistent, OS-managed
```

0.6 SSH vs. HTTPS+PAT vs. Password — Which Do I Actually Use?

Method	How it authenticates	Best for
SSH key	A cryptographic keypair — no password prompt at all once set up	Your own laptop/desktop, daily development — the default recommendation for most people
HTTPS + PAT	The PAT is typed (or cached) in place of a password	Corporate networks that block SSH's port 22; CI/CD runners; quick one-off clones on a machine that isn't yours
Plain password	—	No longer works for Git operations on GitHub (since Aug 2021) — password login still works for the website itself, just not <code>git clone</code> / <code>push</code>

0.7 Verify the Full Chain

Prove the whole setup works end to end before trusting it:

```
# 1. Create a new repository on github.com (use its "New repository" button)
# 2. Clone it using the SSH URL shown on that repo's page
git clone git@github.com:username/repo-name.git
cd repo-name

# 3. Make a real change
echo "# My first repo" > README.md
git add README.md
git commit -m "Initial commit"

# 4. Push it
git push origin main

# 5. Refresh the repo page on github.com – your commit should be there
```

0.8 Common First-Time Errors

Error	What it means
<code>Permission denied (publickey)</code>	Your SSH key isn't loaded into the agent, or isn't added to your GitHub account yet — redo Section 0.4, steps 2–4.
<code>remote: Support for password authentication was removed</code>	You're using HTTPS and typed your GitHub password — GitHub wants a PAT instead (Section 0.5).
<code>fatal: refusing to merge unrelated histories</code>	You're pulling into a repo that was initialized separately from the remote — usually happens the very first time, when both sides have an initial commit. Fix once with <code>git pull origin main --allow-unrelated-histories</code> .
Branch named <code>master</code> instead of <code>main</code>	Older Git versions default to <code>master</code> ; GitHub now defaults new repos to <code>main</code> . Set <code>git config --global init.defaultBranch main</code> once, so every new local repo matches.

01Git Internals

Everything in Sections 02–04 describes Git's *porcelain* — the user-facing commands. This section explains the *plumbing* underneath: why a commit is "just a hash," why branches are "cheap," and why `git add` does more than it looks like it does. Understanding this layer is what turns memorized commands into intuition.

1.1 The Object Model

Everything Git tracks is stored as one of four object types inside `.git/objects`, each addressed by the SHA-1 (or SHA-256, on newer repos) hash of its own content. This is called **content-addressable storage**: the name of an object *is* a hash of what's inside it. Change one byte, and the object gets a completely new address — nothing is ever silently overwritten.

Object	What it stores	Analogy
<code>blob</code>	Raw file content only — no filename, no permissions.	The bytes inside one file, at one point in time.
<code>tree</code>	A directory listing: filenames, modes, and pointers to blobs or other trees.	A folder snapshot.
<code>commit</code>	A pointer to one tree, plus author, message, timestamp, and pointer(s) to parent commit(s).	A labeled snapshot of the whole project at one moment.
<code>tag</code>	(Annotated tags only) A named, signed pointer to a commit, with its own message.	A sticky note that never moves.



Why this matters: a commit doesn't store a diff — it stores a full snapshot via its tree. Git computes the diff you see in `git show` or `git log -p` on the fly, by comparing two trees. This is why checking out any commit is equally fast regardless of how deep in history it is.

1.2 Inspecting Objects Directly (Plumbing Commands)

```
# Hash arbitrary content without storing it
echo "hello" | git hash-object --stdin

# Hash AND store it as a blob object
echo "hello" | git hash-object -w --stdin

# Read back the raw content of any object by hash
git cat-file -p <hash>

# See an object's type
git cat-file -t <hash>

# List the contents of a tree object
git ls-tree HEAD

# Resolve any ref/shorthand to its full hash
git rev-parse HEAD
git rev-parse HEAD~2
git rev-parse main@{yesterday}
```

Hands-on: run `git cat-file -p HEAD` in any repo. You'll see the commit object's raw text: a `tree` line, a `parent` line, author/committer lines, and the message. Then run `git cat-file -p` on the hash after `tree` — you're now looking at the directory snapshot itself. This is the entire object model, visible with two commands.

1.3 The `.git` Directory

Path	Purpose
<code>.git/objects/</code>	Every blob, tree, commit, and annotated tag — content-addressed, loose or packed.
<code>.git/refs/heads/</code>	One file per local branch. Each file's entire content is a single 40/64-char commit hash.
<code>.git/refs/tags/</code>	Same idea, for tags.
<code>.git/HEAD</code>	Usually contains <code>ref: refs/heads/<branch></code> — a pointer to a pointer. In detached HEAD state, it holds a raw commit hash instead.
<code>.git/index</code>	The real staging area — a binary file listing every staged path, its blob hash, and metadata. <code>git add</code> writes to this file.
<code>.git/config</code>	Repo-local settings (remotes, aliases, user overrides).
<code>.git/packed-refs</code>	Compressed refs, written during housekeeping.
<code>.git/logs/</code>	The reflog — a history of where every ref has pointed, kept independently of commit history.

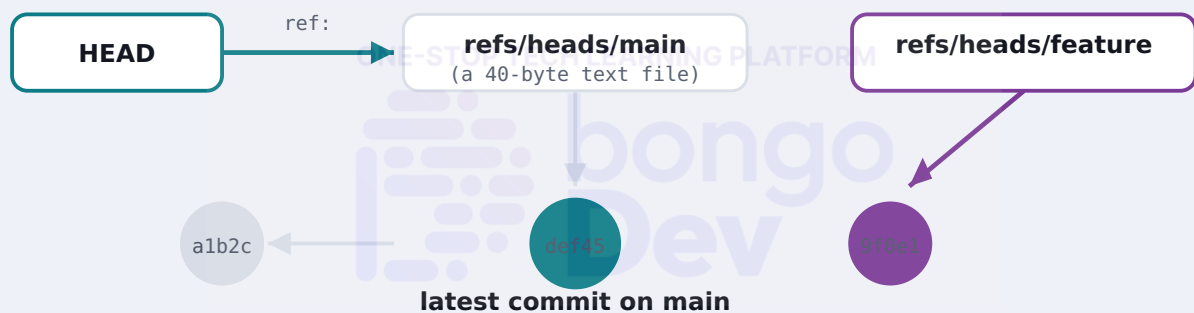


Fig. 1.3 — HEAD points to a branch ref, which points to a commit. Switching branches just rewrites what HEAD points to — nothing on disk is duplicated. A second branch is simply another 40-byte file pointing at a different commit.

Why branches are "cheap": creating a branch is `git branch feature` — which does nothing more than write one 40-byte hash into a new file at `.git/refs/heads/feature`. No files are copied, no history is duplicated. This is the mechanical reason branching in Git costs almost nothing, versus copying an entire directory tree in older SCM tools.

1.4 Packfiles & Garbage Collection

New objects are written as individual "loose" files. Periodically (or via `git gc`), Git compresses many loose objects into a single **packfile**, using delta compression between similar objects — this is most of why a `.git` folder is far smaller than the sum of every full snapshot would suggest.

This is also why `git reflog`-recoverable commits aren't permanent: an unreferenced object survives until garbage collection actually runs (default: unreachable objects older than 90 days, or sooner via `git gc --prune=now`). Until then, "deleted" work is really just unreferenced, not gone.

1.5 What `git add` Actually Does

- 1 Git reads the file's current content and computes its hash.
- 2 Git writes that content as a new `blob` object into `.git/objects` (if it doesn't already exist there).
- 3 Git updates `.git/index` to record: this path → this blob hash.

No tree or commit object exists yet — those are only built at `git commit` time, from whatever is currently sitting in the index. This is why `git diff` (working dir vs. index) and `git diff --staged` (index vs. last commit) are comparing genuinely different pairs of snapshots, not just "before/after add."

ONE-STOP TECH LEARNING PLATFORM



02Core Concepts

2.1 Source Code Management

Every team writing code needs a system to track changes, coordinate multiple contributors, and recover from mistakes. That system is SCM, and it comes in two flavors.

Centralized (CVCS) — one central server holds the repository; every contributor talks directly to it. Almost every operation needs a network connection, and the server is a single point of failure. Examples: SVN, Perforce.

Distributed (DVCS) — every contributor clones the full repository, history and all. Most operations work offline, and any clone can restore the project. Examples: Git, Mercurial.

Why it matters for DevOps: CI/CD pipelines need fast, reliable access to full history (for `git bisect` or rollback). A distributed model means build agents and engineers all work from local clones, without depending on one fragile central server.

2.2 Git's Three-Stage Architecture (User View)

Stage	Description
Working Directory	The files you see and edit on disk.
Staging Area	A holding zone for changes marked ready via <code>git add</code> . Backed by <code>.git/index</code> — see Section 1.5.
Repository	Permanent committed history, stored in <code>.git/objects</code> — see Section 1.1.

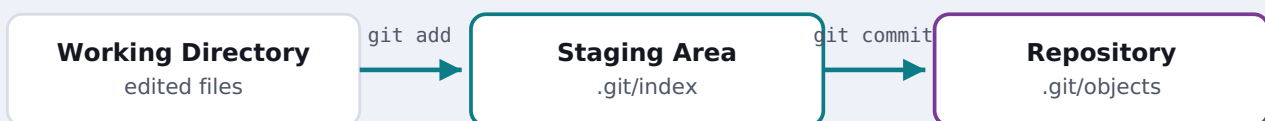


Fig. 2.2 — The path every change takes: edit on disk, mark ready with `git add`, commit permanently into history.

2.3 Key Terms

- **Repository** — a folder tracked by Git, containing all files plus full change history.
- **Commit** — a saved snapshot, identified by a content-derived hash (Section 1.1). Change one character of tracked content and the hash changes completely — tamper-evident by construction.
- **Tag** — a fixed label pointing at one commit (e.g. `v1.2.0`). Unlike a branch, it doesn't move.

- **Branch** — a movable pointer to a commit (literally a 40-byte file, Section 1.3), representing an independent line of development.
- **HEAD** — a pointer to whatever commit you currently have checked out (usually indirectly, via a branch — Section 1.3).

2.4 Branching, Merging & Beyond

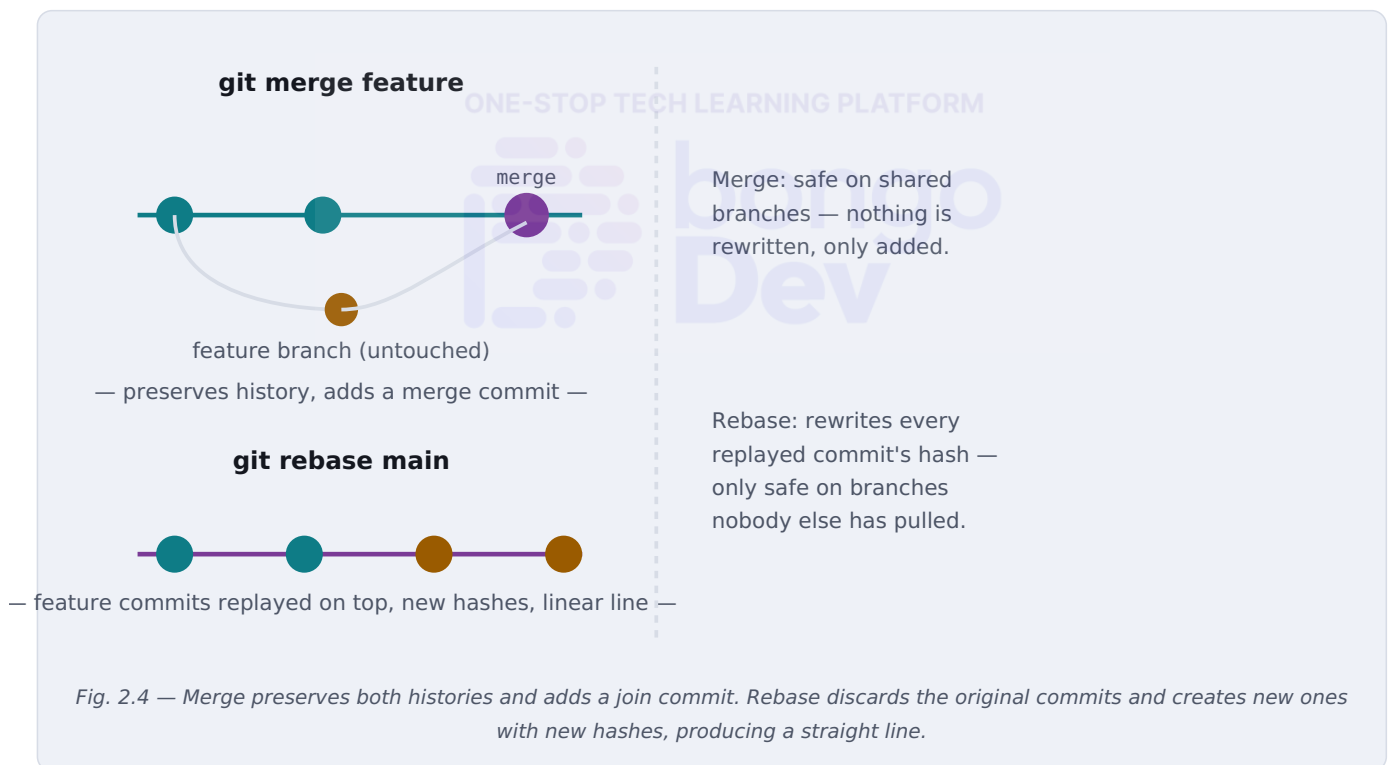
Push sends local commits to a remote. **Pull** fetches and merges remote commits into your current branch. **Fetch** downloads without merging, so you can inspect first.

Cherry-pick (`git cherry-pick <commit>`) applies one specific commit from another branch, without merging the whole branch.

Stashing temporarily shelves uncommitted changes: `git stash` → `git stash list` → `git stash apply` (keeps it) or `git stash pop` (removes it) → `git stash drop` (discards it).

Rebasing replays commits onto a different base, producing linear history instead of a merge bubble.

Rule of thumb: never rebase commits that have already been pushed and shared — it rewrites hashes (Section 1.1: new content → new hash), breaking history for anyone who already has the old commits.



Squashing combines multiple commits into one — via interactive rebase (`git rebase -i HEAD~N`, mark commits `squash` / `fixup`) or a host's "squash and merge" on a pull request.

2.5 Undoing Changes — Choosing the Right Tool

Command	Effect	Use when
<code>git restore <file></code>	Discards uncommitted changes in the working directory	You edited a file and want to throw it away
<code>git reset <file></code>	Unstages a file; changes are kept	You staged something by mistake
<code>git reset --mixed <commit></code>	Moves branch pointer back; keeps changes unstaged	Redo staging/committing from an earlier point
<code>git reset --hard <commit></code>	Moves branch pointer back; discards all changes	Fully discard commits and edits — destructive
<code>git revert <commit></code>	Creates a new commit that undoes a previous one	Undo something already pushed or shared

`revert` is the safe, collaborative option — it never rewrites history. `reset --hard` is fast but dangerous on shared branches. Either way, the old commit object usually isn't gone (Section 1.4) — it's recoverable via `git reflog` until garbage collection runs.

ONE-STOP TECH LEARNING PLATFORM



03Command Cheatsheet

Common Git & GitHub Terms, Clearly Compared

These are the words that get thrown around interchangeably in tutorials and cause the most confusion for anyone new to the ecosystem. Each pair below looks similar but means something distinct.

Term	What it actually does	Use it when
<code>git merge</code>	Combines two branches' histories by creating a new merge commit. Nothing is rewritten.	Integrating a finished feature via a PR — safe on branches others share.
<code>git rebase</code>	Replays your commits onto a new base commit, giving each one a brand-new hash.	Keeping your own not-yet-shared feature branch up to date with main.
Squash	Combines many commits into one, either via interactive rebase or a host's "Squash and merge" button.	Cleaning up a noisy PR ("fix typo" ×12) into one atomic commit before it lands on main.

Term	What it actually does
<code>git clone</code>	Copies an entire remote repository to your machine for the first time — including full history.
<code>git fetch</code>	Downloads new commits and refs from a remote, but touches nothing in your working branch.
<code>git pull</code>	<code>fetch</code> immediately followed by a <code>merge</code> (or <code>rebase</code> , with <code>--rebase</code>) into your current branch.

Term	What it actually does
Fork	A full copy of someone else's repository into your own GitHub account. Used when you don't have push access to the original.
Branch	A pointer to a commit inside the <i>same</i> repository. Used when you do have push access (teammates on the same project).

Term	What it actually does
<code>origin</code>	The default name Git gives to the remote you cloned from. Just a label — not a special keyword.
<code>upstream</code>	A second remote you add by hand, conventionally pointing to the original repo you forked from — lets you pull in changes from the project you forked.

Term	What it actually does
HEAD	Normally points to your current branch, which points to a commit — see Section 1.3.
Detached HEAD	HEAD points directly at a commit instead of a branch (happens after checking out a raw hash or tag). New commits made here can become unreachable if you don't branch off immediately.

Term	What it actually does
SSH key	A cryptographic keypair proving your identity — no password prompt once configured. Covered in Getting Started, above.
Personal Access Token (PAT)	A generated string used as a password substitute for HTTPS-based Git operations. Covered in Getting Started, above.
GitHub password	Only works for logging into the website — no longer accepted for <code>git clone</code> / <code>push</code> since 2021.

Term	What it actually does
Untracked	A file Git has never seen before — not in any past commit, not staged.
Unstaged / Modified	A tracked file with changes that haven't been added to staging yet.
Staged	Changes added via <code>git add</code> , ready to be included in the next commit.
Tracked	Any file Git is watching at all — covers staged, modified, and unmodified-but-committed files alike.

Term	What it actually does
Local branch	e.g. <code>main</code> — yours, moves forward every time you commit.
Remote-tracking branch	e.g. <code>origin/main</code> — a read-only bookmark of where the remote's branch was as of your last <code>fetch</code> . It never moves from a local commit — only from <code>fetch</code> , <code>pull</code> , or <code>push</code> .

Setup

Command	Description
<code>git config --global user.name "<name>"</code>	Set global commit author name
<code>git config --global user.email "<email>"</code>	Set global commit author email
<code>git config --global init.defaultBranch main</code>	Standardize the default branch name for new repos
<code>git config --global fetch.prune true</code>	Auto-remove local refs to branches deleted on the remote
<code>git config --list</code>	Show current configuration
<code>git init</code>	Initialize a new repository
<code>git clone <repo-url></code>	Clone an existing repository locally

Remotes

Command	Description
<code>git remote -v</code>	Show configured remote URLs
<code>git remote add origin <url></code>	Link local repo to a remote
<code>git remote remove origin</code>	Remove a remote link
<code>git fetch</code>	Download remote commits/refs without merging

Staging & Committing

Command	Description
<code>git status</code>	Show working directory / staging state
<code>git add <file></code> / <code>git add .</code>	Stage a specific file / stage all changed files
<code>git commit -m "<message>"</code>	Commit staged changes
<code>git commit --amend</code>	Edit the most recent commit
<code>git commit --fixup <commit></code>	Create a commit flagged as a fix for an earlier one — pairs with <code>rebase --autosquash</code> below
<code>git restore <file></code>	Discard uncommitted changes
<code>git reset <file></code>	Unstage a file, keep its edits

Branching

Command	Description
<code>git branch</code>	List local branches
<code>git branch --merged</code> / <code>--no-merged</code>	List branches already merged / not yet merged into current — good for cleanup audits
<code>git switch <branch></code> / <code>git switch -c <branch></code>	Switch to / create-and-switch to a branch
<code>git branch -d <branch></code> / <code>-D</code>	Delete a branch (safe) / force-delete

Merging & Diffing

Command	Description
<code>git merge <branch></code>	Merge the named branch into current
<code>git merge --abort</code>	Cancel an in-progress merge
<code>git log --merge</code>	List commits causing a conflict
<code>git diff</code> / <code>git diff --staged</code>	Show unstaged / staged-but-uncommitted changes

History & Recovery

Command	Description
<code>git log --oneline --graph -all</code>	Visualize branch/merge structure
<code>git log --<path></code>	Commits that touched a file/folder
<code>git log --author="<name>" / --grep="<text>"</code>	Filter by author / commit message content
<code>git log --since=... --until=...</code>	Filter by date range
<code>git blame <file></code>	Who last modified each line
<code>git show <commit></code>	Show details/diff of a commit
<code>git reflog</code>	The safety net. Logs every place HEAD/branches have pointed, independent of commit ancestry — recovers commits "lost" to a bad reset, rebase, or branch deletion, until garbage collection runs (Section 1.4).

Recovering "lost" work: after a bad `git reset --hard`, run `git reflog`, find the entry just before the mistake, then `git reset --hard HEAD@{n}` or `git branch rescue HEAD@{n}`. The commit was never deleted — only unreferenced.

Push, Pull & Stash

Command	Description
<code>git push -u origin <branch></code>	Push, and track the remote branch
<code>git push --force-with-lease</code>	Force-push safely — refuses if someone else has pushed since your last fetch (unlike plain <code>--force</code>)
<code>git pull --rebase origin <branch></code>	Fetch, then rebase local commits on top
<code>git stash</code> / <code>git stash pop</code>	Save uncommitted changes / reapply and remove from stash

Rebase, Cherry-Pick & Tags

Command	Description
<code>git rebase -i <commit></code>	Reorder, squash, edit, or drop commits
<code>git rebase -i --autosquash</code>	Auto-orders <code>fixup!</code> / <code>squash!</code> commits (from <code>commit --fixup</code>) next to their targets
<code>git rebase --continue</code> / <code>--abort</code>	Continue after resolving, or cancel entirely
<code>git cherry-pick <commit></code>	Apply a single commit from another branch
<code>git cherry-pick -n <commit></code>	Cherry-pick without auto-committing — review/selectively stage first
<code>git tag -a <name> -m "<msg>"</code>	Create an annotated tag
<code>git push origin --tags</code>	Push all local tags

Debugging & Housekeeping

Command	Description
<code>git bisect start</code> / <code>good</code> / <code>bad</code> / <code>reset</code>	Binary-search for the commit that introduced a bug
<code>git bisect run <script></code>	Automate the good/bad check with a script that exits 0 (good) or non-zero (bad)
<code>git clean -f</code> / <code>-fd</code>	Remove untracked files / untracked files & directories. Irreversible — combine with <code>-n</code> (dry run) first.
<code>git worktree add <path></code> <code><branch></code>	Check out a branch into a separate folder — no stashing needed for a quick hotfix
<code>git submodule add <url></code> <code><path></code>	Nest another repository inside this one at a pinned commit
<code>git submodule update --init</code> <code>--recursive</code>	Populate submodules after cloning a repo that has them
<code>git format-patch -1</code> <code><commit></code> / <code>git am <patch></code>	Export a commit as an email-style patch file / apply one — used where PRs aren't the workflow (e.g. mailing-list-based projects)
<code>git rm --cached <file></code>	Stop tracking a file without deleting it locally
<code>.gitignore</code>	Patterns Git should never track (e.g. <code>.env</code> , <code>node_modules/</code>). Only affects untracked files — see Q16.

Git Hooks

Hooks are scripts in `.git/hooks/` that run automatically at specific points in the Git lifecycle — never committed by default (they're local), but commonly distributed via a tool like `husky` or `pre-commit` so every clone gets the same hooks.

Hook	Fires	Common use
<code>pre-commit</code>	Before a commit is created	Linting, formatting, secret-scanning the staged diff
<code>commit-msg</code>	After the message is written, before the commit is finalized	Enforce conventional-commit format
<code>pre-push</code>	Before a push leaves the machine	Run the test suite; block pushes to protected branches locally

Signed Commits

Anyone can set `git config user.name` to any string — nothing verifies identity by default. Signing proves a commit's author cryptographically.

```
# Generate an SSH signing key
ssh-keygen -t ed25519 -C "your_email@example.com"

# Configure Git to sign with it
git config --global gpg.format ssh
git config --global user.signingkey "~/ssh/id_ed25519.pub"
git config --global commit.gpgsign true

# Upload the PUBLIC key to GitHub as a "Signing Key" (not an auth key)
```

Once configured, every commit carries a "Verified" badge on GitHub once the matching public key is registered there.

Git LFS (Large File Storage)

Git is optimized for text diffs, not large binaries — a repo with committed video/design files bloats fast because every version of a binary is stored in full. LFS replaces large files in history with lightweight text pointers, storing the actual bytes on a separate LFS server.

```
git lfs install
git lfs track "*.psd" "*.mp4"
git add .gitattributes # the tracking rule itself must be committed
git add largefile.mp4
git commit -m "Add asset via LFS"
```

GitHub Workflows

```
# SSH auth
ssh-keygen
cat ~/.ssh/<key_name>.pub          # add under GitHub → Settings → SSH keys
ssh -T git@github.com              # verify

# Fork & sync
git remote add upstream https://github.com/<original-owner>/<repo>.git
git fetch upstream
git merge upstream/main
```

```
# Minimal CI workflow – runs tests on every push or PR to main
name: CI/CD Pipeline
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
```

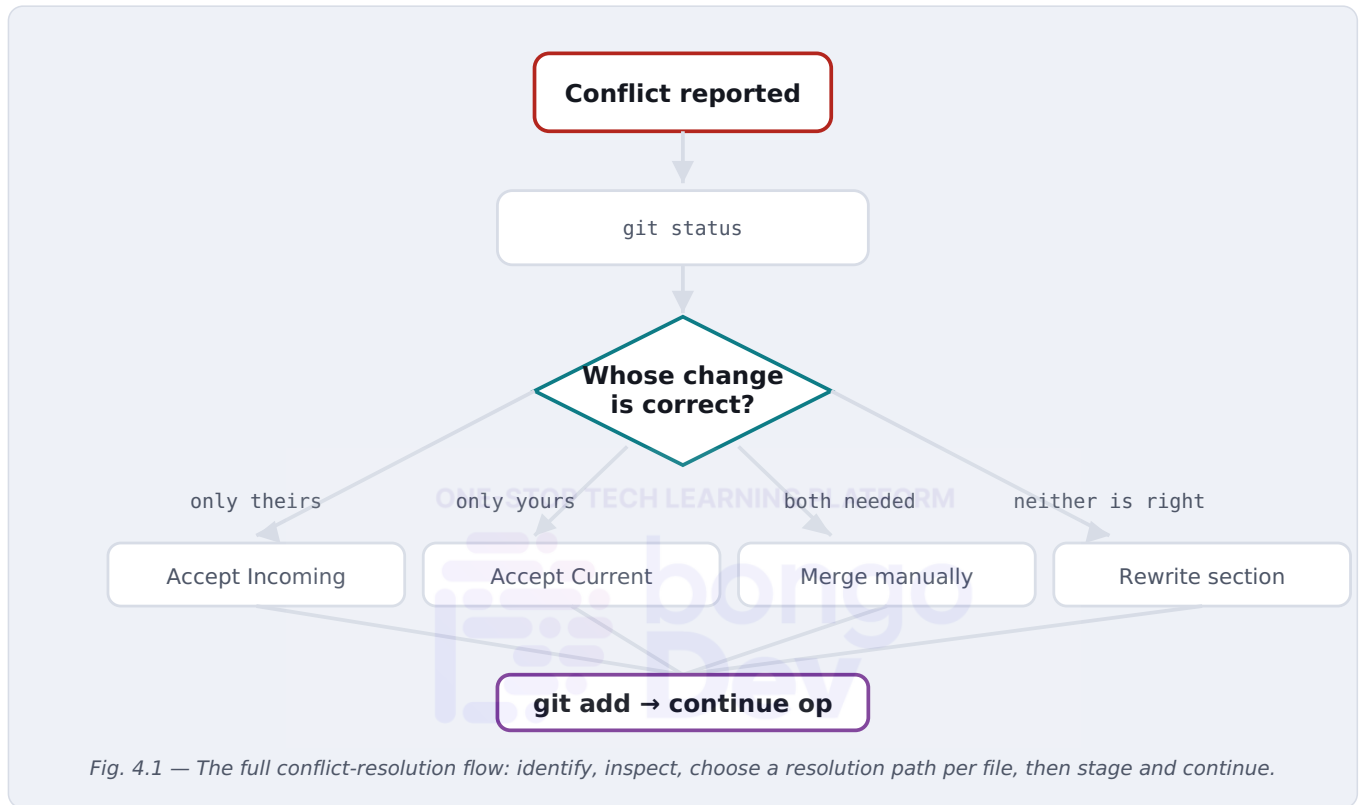
ONE-STOP TECH LEARNING PLATFORM



04Merge Conflict Playbook

A conflict occurs when Git can't automatically reconcile changes because the same lines of the same file were modified differently on two branches. Git merges everything it can figure out and stops only where it can't — it never guesses. Conflicts most commonly surface during merge, rebase, cherry-pick, and pull.

4.1 The Decision Tree



- 1 **git status** — see which files are conflicted.
- 2 Open each file. Read both sides — don't pick blindly.

Situation	Action
Only theirs is right	Keep the incoming change
Only yours is right	Keep the current change
Both are needed	Manually merge the logic
Neither is right	Rewrite the section

- 3 Remove all conflict markers (`<<<<<<` , `=====` , `>>>>>>`).
- 4 Stage the resolved file: `git add <file>` . Repeat for any remaining files.
- 5 Continue the operation — `git commit` (merge), `git rebase --continue` , or `git cherry-pick --continue` .
- 6 Run the test suite / build before pushing.

If a conflict is too tangled, or you resolved it wrong — abort and start over rather than pushing a half-fixed merge: `git merge --abort` · `git rebase --abort` · `git cherry-pick --abort` .

4.2 Reading Conflict Markers

```
<<<<<< HEAD
console.log("current branch version");
=====
console.log("incoming branch version");
>>>>>> feature-branch
```

Everything between `<<<<<< HEAD` and `=====` is your current branch's version; between `=====` and `>>>>>> feature-branch` is the incoming version. Edit down to what the code should actually be, then delete all three marker lines.

Leaving a marker in place is a common mistake — Git considers the conflict resolved once you `add` it, even though the file may now be syntactically broken.

VS Code shortcut — conflicted files show inline "Accept Current," "Accept Incoming," "Accept Both," and "Compare Changes" buttons above each block.

4.3 Binary and Rename Conflicts

The text-marker model above only works for line-based text conflicts. Two other conflict shapes behave differently:

- **Binary file conflicts** — Git can't show inline markers inside a binary. `git status` reports the file as conflicted; you must manually choose a version with `git checkout --ours <file>` or `--theirs <file>` , then `git add` it.
- **Rename conflicts** — if one branch renames a file and another edits the original path, Git's rename detection may not link them automatically. `git status` reports it as "both added" or similar; resolve by manually deciding which path and content should survive.

4.4 Prevention Checklist

- Pull/rebase frequently — the longer a branch diverges, the more surface area for conflicts.

- Keep PRs small and short-lived — a 3-file, 1-day PR rarely conflicts; a 40-file, 3-week PR almost always does.
- Communicate on shared files — a 30-second heads-up prevents a 30-minute resolution.
- Rebase onto main before opening a PR, not after — surface conflicts while context is fresh.
- Never rebase or force-push a branch others have already pulled.

4.5 Hands-On Exercise

Run this in a scratch folder — safe, self-contained, about five minutes.

```
# 1. Set up a fresh repo with one file
mkdir conflict-lab && cd conflict-lab
git init
echo "Hello from main" > greeting.txt
git add greeting.txt
git commit -m "Initial commit"

# 2. Create a branch and change the file
git checkout -b feature-branch
echo "Hello from feature-branch" > greeting.txt
git commit -am "Update greeting on feature-branch"

# 3. Switch back to main, change the SAME line differently
git checkout main
echo "Hello from main, updated" > greeting.txt
git commit -am "Update greeting on main"

# 4. Attempt the merge – this WILL conflict
git merge feature-branch
```

```
git status
cat greeting.txt

# edit the file by hand to the final version, e.g.:
echo "Hello from main, updated and merged with feature-branch" > greeting.txt

git add greeting.txt
git commit -m "Merge feature-branch, resolve greeting conflict"
git log --oneline --graph --all
```

Extend it — repeat with `git rebase feature-branch` instead of merge, and compare the conflict flow and resulting history (`rebase --continue` vs. `commit`). Then try `git merge --abort` mid-conflict and confirm the repo returns cleanly. Then run `git reflog` and locate every step you just took — this ties Section 1.4 directly to what you just did. Delete `conflict-lab` when done.

05Enterprise Repository Architecture

Commands and conflict resolution are mechanics. This section is judgment: how real teams choose a branching model, structure a repository, and gate what reaches production.

5.1 Branching Strategies — The Complete Reference

The choice of branching strategy is architectural, not cosmetic — it dictates deployment frequency, review overhead, and how much automated testing discipline the team needs. This section covers all six strategies a working engineer will actually encounter, from a solo prototype to a versioned enterprise product, each with its own commit graph, workflow, and real trade-offs — zero prior branching-strategy knowledge assumed.

Strategy	Team Fit	Isolation	Deployment Cadence	Naming Example
Main-Only	Solo / prototype	None	Ad-hoc	N/A (direct commits)
Feature Branching	Small/medium teams	Moderate	Regular	<code>feature/alice/add-login-button</code>
GitHub Flow	SaaS / CI/CD teams	High	Immediate, continuous	<code>alice/fix-api-endpoint</code>
Trunk-Based	High-discipline, mature test coverage	Low (relies on feature flags, not branches)	Multiple per day	<code>fix/1234</code>
Gitflow	Large, scheduled-release enterprise	Very high	Scheduled / LTS	<code>feature/john/1234/add-login</code>
Release Branching	Versioned/packaged software	Version-isolated	Versioned / LTS	<code>release/1.2.0</code>

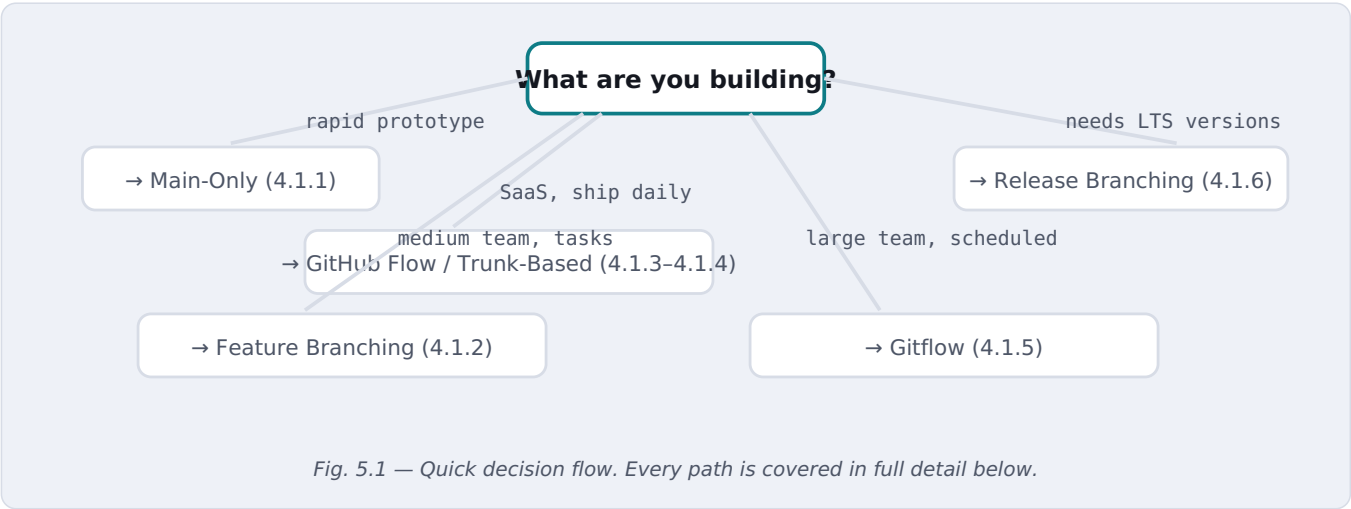


Fig. 5.1 — Quick decision flow. Every path is covered in full detail below.

Trunk-Based & Feature Flags: Trunk-Based Development keeps branch lifetimes to hours, not days — which only works because incomplete features are hidden behind runtime feature flags rather than isolated in long-lived branches. Without flags, trunk-based development simply breaks main constantly. This is the piece most branching-strategy summaries skip — see 4.1.4.

5.1.1 Main-Only

Every commit lands directly on `main`. There is no isolation between "in progress" and "shipped" — the working tree *is* the product at every moment.

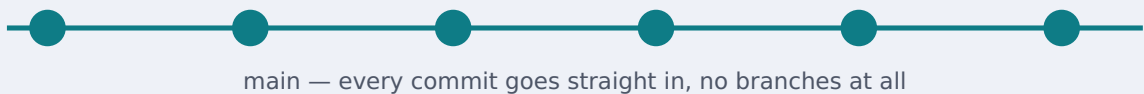


Fig. 5.1.1 — Main-Only: a single, ever-growing line.

```
git add .
git commit -m "feat: add landing page"
git push origin main
```

Pros	Cons
Simplest possible workflow; zero merge conflicts to manage	No isolation — a half-finished change is live the moment it's pushed
Nothing to learn beyond <code>add</code> / <code>commit</code> / <code>push</code>	No safe way to experiment or roll back a single feature independently

Use it for: solo prototypes, throwaway scripts, proof-of-concept spikes. **Avoid it for:** anything more than one contributor, or any code that needs to stay deployable at all times.

5.1.2 Feature Branching

Each unit of work — a feature, a bugfix — gets its own short-lived branch off `main`, and merges back once complete.

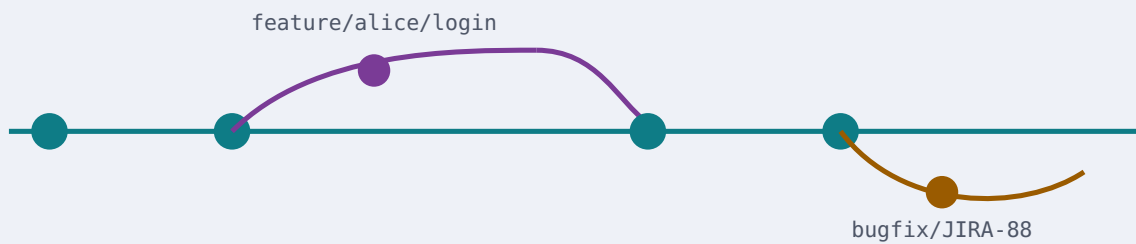


Fig. 5.1.2 — Short-lived task branches fork off main and merge back once reviewed.

```
git switch -c feature/alice/add-login-button main
# ... work, commit ...
git push -u origin feature/alice/add-login-button
# open a PR, review, then merge into main
```

Pros	Cons
Clear isolation of changes; easy to discard a buggy feature without touching main	Risk of "merge hell" if a branch lives too long without syncing with main
Natural fit for PR-based code review	Requires discipline to keep branches short-lived

Use it for: the default choice for most small/medium teams shipping regularly. **Avoid it for:** teams needing multiple parallel release versions in support simultaneously (use Release Branching instead).

5.1.3 GitHub Flow

A stripped-down feature-branching model built for continuous delivery: branch, commit, open a PR early, merge to `main` the moment it's approved and green, deploy immediately.



every merge to main is deployed immediately — branches live hours, not days

Fig. 5.1.3 — GitHub Flow: frequent, small branches, each shipped the moment it merges.

```
git switch -c alice/fix-api-endpoint main
git push -u origin alice/fix-api-endpoint
# open PR immediately (even in draft) → CI runs → review → merge → auto-deploy
```

Pros	Cons
Perfectly optimized for CI/CD; encourages small, incremental, low-risk shipments	Lacks structure for long-term pre-release testing or versioned support
Only one long-lived branch (<code>main</code>) to reason about	Requires solid CI and a low-friction deploy pipeline to be safe

Use it for: cloud-native SaaS shipping many times a day. **Avoid it for:** shrink-wrapped/installed software that needs a stabilization window before release.

5.1.4 Trunk-Based Development

The most aggressive integration model: everyone commits to (or merges into) `main` / `trunk` at least daily, often multiple times a day. Branches, if they exist at all, live for hours. Incomplete work ships to trunk constantly — **hidden behind feature flags** rather than isolated in a branch.

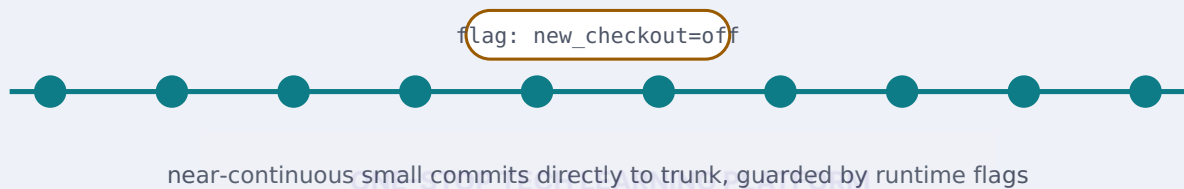


Fig. 5.1.4 — Trunk-Based: almost no branch structure at all; safety comes from flags and tests, not isolation.

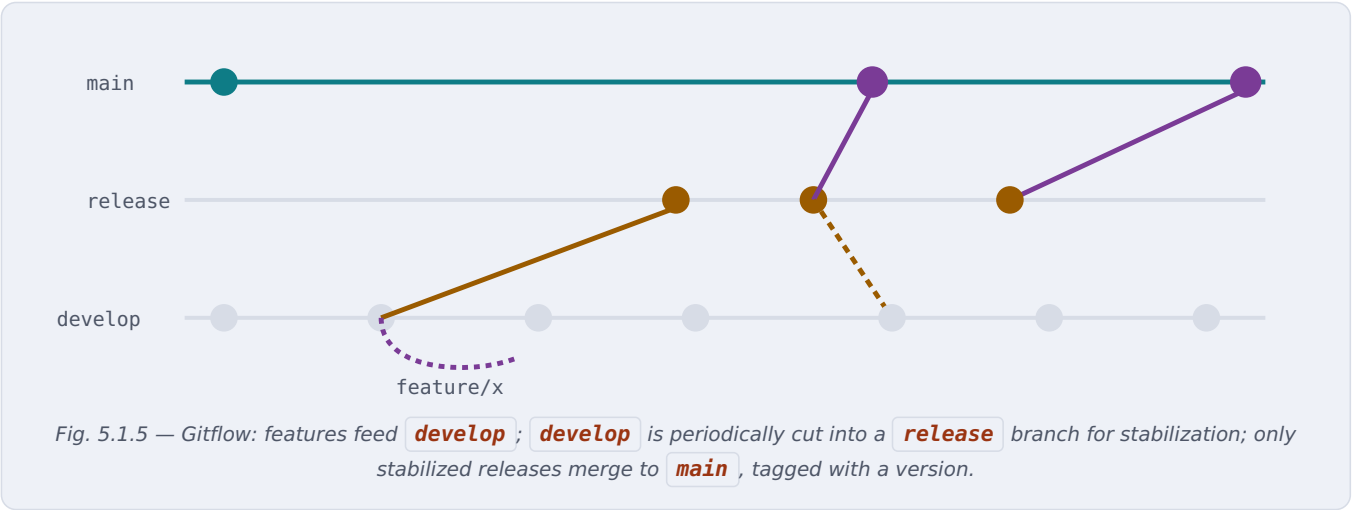
```
# Feature shipped but disabled by default
if (featureFlags.isEnabled("new_checkout")) {
  renderNewCheckout();
} else {
  renderLegacyCheckout();
}
// merged to trunk same day, flag flips on later once verified
```

Pros	Cons
Minimal branch complexity; forces frequent, small code reviews	Requires mature automated test coverage — there's no long-lived branch to catch problems first
Fastest possible integration; near-zero merge drift	High risk of trunk instability without disciplined flag hygiene and CI gating

Use it for: high-seniority, high-test-maturity teams (this is how Google and Meta operate internally). **Avoid it for:** teams without strong CI or without a feature-flag system already in place — see interview Question 19.

5.1.5 Gitflow

A structured, multi-lane model with dedicated permanent branches for integration, release stabilization, and production — designed for scheduled, versioned releases.



```
git switch -c feature/john/1234/add-login develop
# ... work, PR, merge into develop ...

# When ready to stabilize a release:
git switch -c release/1.3.0 develop
# bugfixes only on release/1.3.0 from here
git switch main && git merge --no-ff release/1.3.0
git tag -a v1.3.0 -m "Release 1.3.0"
git switch develop && git merge --no-ff release/1.3.0
```

Pros	Cons
Highly organized; dedicated paths for releases and emergency hotfixes	Process-heavy; generally too slow for rapid SaaS continuous delivery
Clear separation between "in progress," "stabilizing," and "live"	More branches to keep synchronized — see the QA/hotfix protocols in Section 5.3

Use it for: complex enterprise projects with strict, scheduled release cycles. **Avoid it for:** small teams or anything needing same-day deploys — the ceremony will slow you down for no benefit.

5.1.6 Release Branching

Once a version ships, it gets its own permanent branch — so that version can keep receiving patches independently of whatever the next version is doing on **main**.

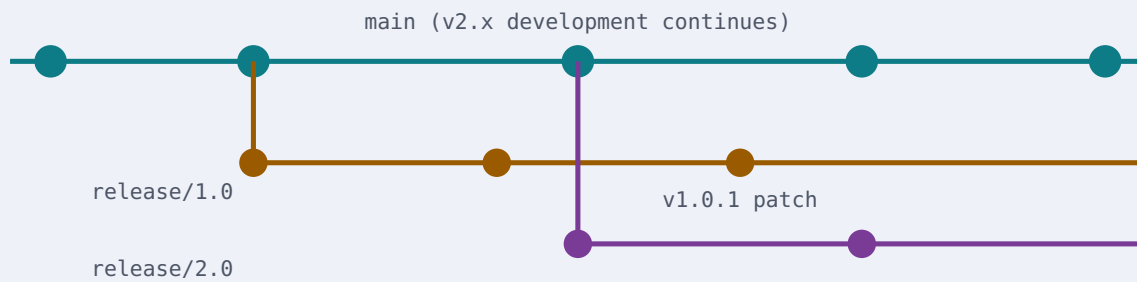


Fig. 5.1.6 — Each shipped version keeps its own long-lived branch, patched independently while main moves on to the next version.

```
git switch -c release/1.0 v1.0.0
# a critical bug is found in 1.0 after 2.0 development has already started
git switch release/1.0
git switch -c hotfix/1.0.1 release/1.0
# ... fix, merge back into release/1.0 ...
git tag v1.0.1
```

Pros	Cons
Explicit version control; enables patching an old release without disrupting new development	High branch proliferation over the life of a product
Ideal for anything customers install and run themselves	Significant administrative overhead — every active version needs its own attention

Use it for: enterprise or packaged software supporting multiple versions in the field simultaneously.

Avoid it for: a single continuously-deployed web service — there's only ever one "version" live, so there's nothing to isolate.

5.2 Repository Scaffold

```

.
├── .github/
│   ├── workflows/          # CI/CD pipeline definitions (ci.yml)
│   ├── issue_template/     # Structured bug/feature YAML forms
│   ├── CODEOWNERS          # Path → reviewer-team routing
│   └── pull_request_template.md
├── .vscode/                # Shared editor configuration
├── docs/
├── k8s/
│   ├── qa/
│   └── prod/
├── scripts/
├── src/
│   ├── client/
│   └── server/
├── terraform/
├── .editorconfig
├── .gitattributes
├── .env.example
├── README.md
├── CONTRIBUTING.md
├── SECURITY.md
├── CODE_OF_CONDUCT.md
└── CHANGELOG.md

```

ONE-STOP TECH LEARNING PLATFORM

The Empty Folder Problem

Git ignores empty directories entirely — a structural folder like `scripts/` with nothing in it yet won't be tracked, which can break deployment scripts expecting that path to exist. Force tracking with a placeholder:

```
find . -type d -empty -not -path "./.git/*" -exec touch {}/.gitkeep \;
```

Configuration Files

File	Purpose
<code>.editorconfig</code>	Enforces consistent indentation, line endings, and charset across VS Code, IntelliJ, Vim, etc. — eliminates formatting-only diff noise in PRs.
<code>.gitattributes</code>	Normalizes line endings (<code>* text=auto eol=lf</code>) for Windows/Linux interop, and can mark paths as <code>linguist-documentation</code> / <code>linguist-vendor</code> / <code>linguist-generated=false</code> so GitHub's language stats reflect real code.
<code>.env.example</code>	A template for required environment variables, committed in place of the real <code>.env</code> (which must be <code>.gitignore</code> d) to prevent secret leaks while still documenting what's needed.

5.3 The Promotion Pipeline (Gitflow-style)

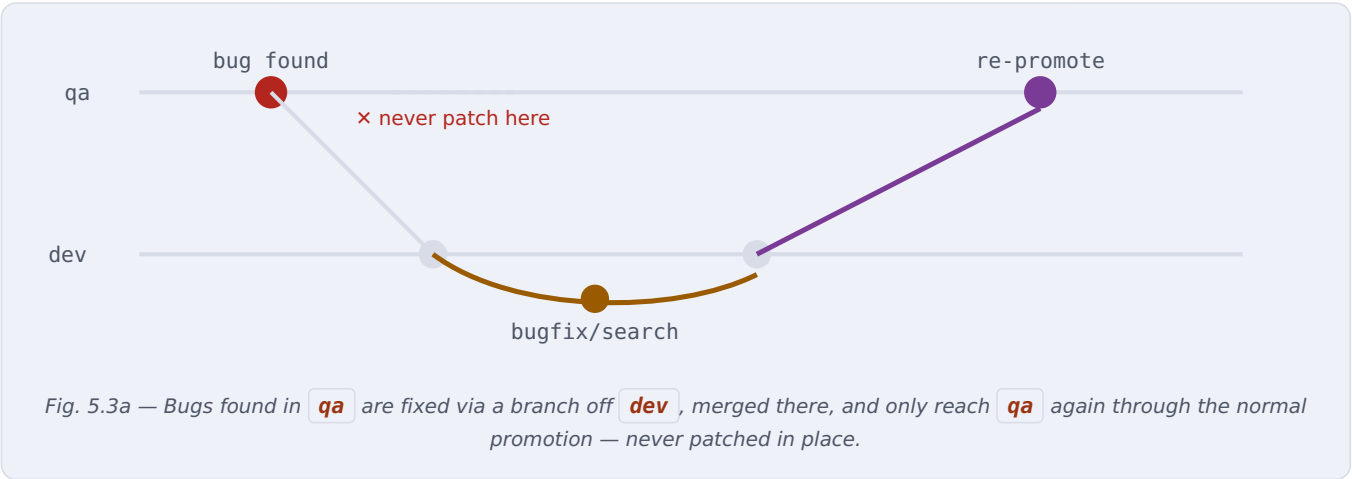
A common enterprise pattern moves code through four permanent, environment-mapped branches:

Branch	Role
dev	Integration — the primary target for all new feature branches.
qa	Functional/manual testing.
ppd (pre-production)	Final smoke and regression testing, closest to real production config.
prod	Default branch — only verified, stable, tagged code.

Feature naming convention: `feature/{author}/{card-number}/{description}` — e.g. `feature/alice/101/add-jwt-auth`.



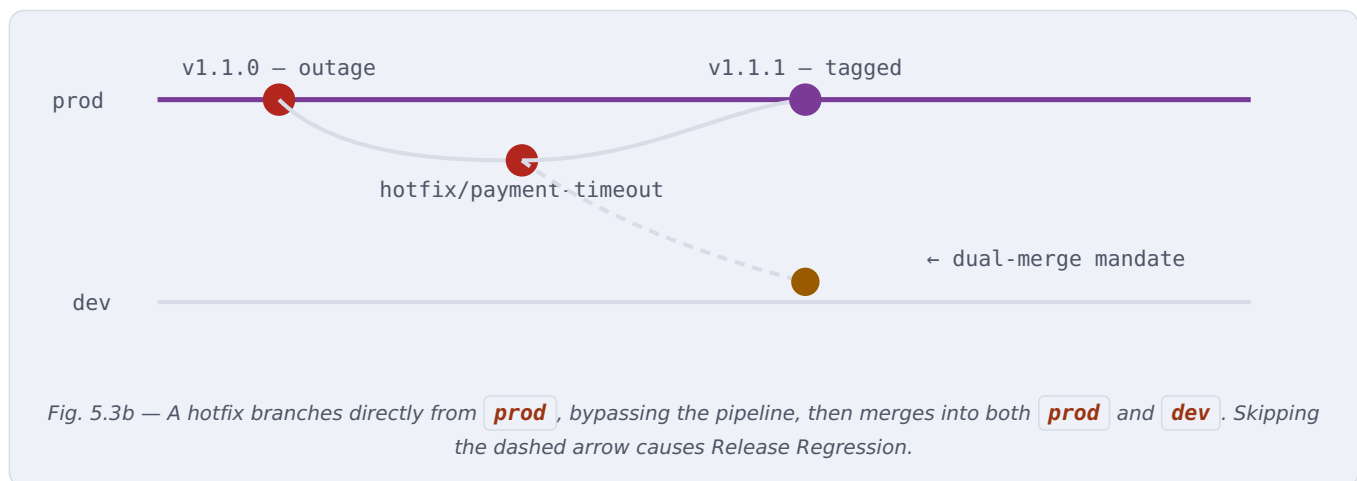
QA Bugfix Protocol



- 1 A defect found in `qa` is **never** patched directly in `qa`.
- 2 Create a `bugfix/` branch originating from `dev`.
- 3 Merge the fix into `dev`, then re-promote `dev` forward into `qa`.

Patching `qa` directly creates an untracked "ghost fix" that gets silently overwritten the next time `dev` is promoted forward — the bug reappears.

Production Hotfix Protocol



- 1 Branch `hotfix/{issue-number}` directly from `prod` — bypassing the normal promotion chain.
- 2 **Dual-Merge Mandate:** once resolved, merge the hotfix into *both* `prod` (immediate fix) and `dev`.

Release Regression: skipping the merge back into `dev` means the next scheduled promotion from `dev` will overwrite the fix and resurrect the exact bug that was just patched in production.

5.4 Semantic Versioning

Adhere to `v[Major].[Minor].[Patch]`:

- **Major** — breaking API changes.
- **Minor** — new features, backward compatible.
- **Patch** — backward-compatible bug fixes or hotfixes.

```
git tag -a v1.1.1 -m "Hotfix: payment timeout"
git push origin --tags
```

5.5 Governance & Security

CODEOWNERS — Delegated Review Authority

A `.github/CODEOWNERS` file maps file paths to reviewer teams, automatically requesting the right reviewers instead of relying on manual tagging:

```
/terraform/*  @infra-team
/src/*        @dev-team
/k8s/*        @devops-engineers
```



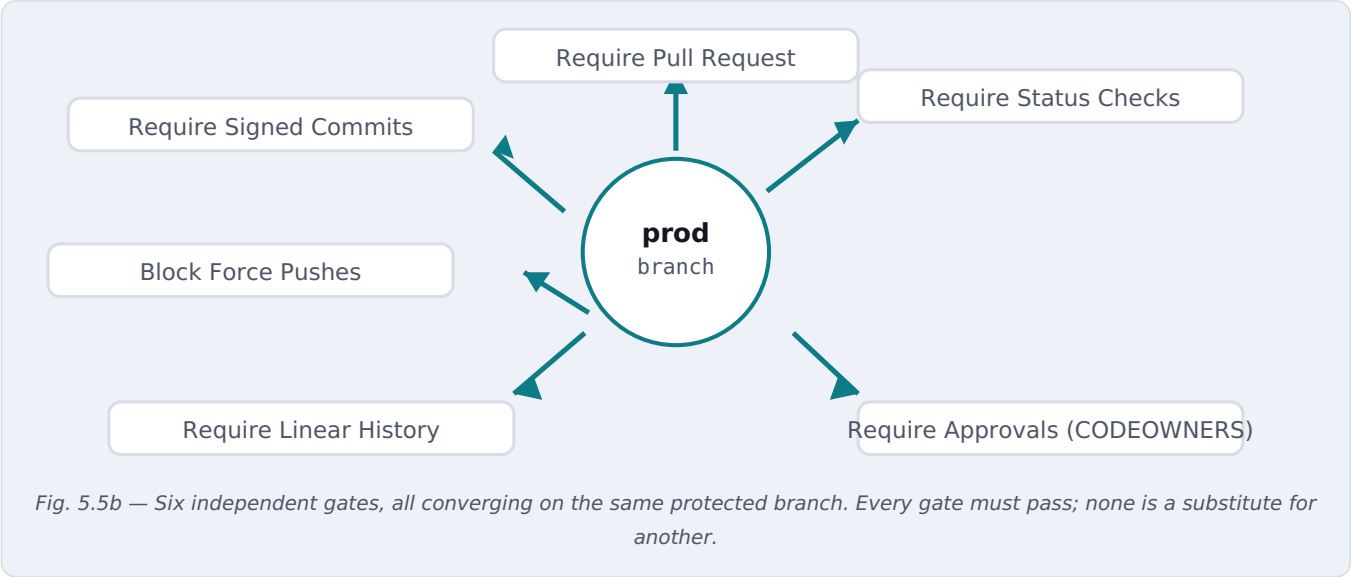
Fig. 5.5a — CODEOWNERS reads every changed path in a PR and routes review requests automatically — no manual tagging needed.

Squash-and-Merge Mandate

Squashing a PR's commits into one on merge to `main` / `prod` collapses noisy iterative history ("fix typo," "trying again") into a single atomic commit — preserving linear history, which is what makes `git bisect` and clean automated rollbacks actually work in practice.

Branch Protection Rulesets (target: `prod` / `main`)

Rule	Effect
Require Pull Request	Disables direct pushes.
Require Status Checks	CI (hygiene/testing) must report green before merge.
Require Approvals	Enforced via CODEOWNERS — the right team must sign off.
Require Signed Commits	Enforces cryptographic "Verified" identity (Section 3, Signed Commits).
Require Linear History	Forces squash or rebase merges only — no merge-commit noise.
Block Force Pushes	Protects the integrity of shared history.



Automated Guardrails

Mechanism	What it does
Dependabot (<code>dependabot.yml</code>)	Scheduled scans of the dependency graph; auto-opens PRs patching vulnerable packages.
Secret Scanning / Push Protection	Rejects a <code>git push</code> outright at the server if it detects a known secret pattern (API key, token) in the payload — distinct from Git LFS's file-size rejection , and not a default local Git behavior — it's a GitHub platform feature.

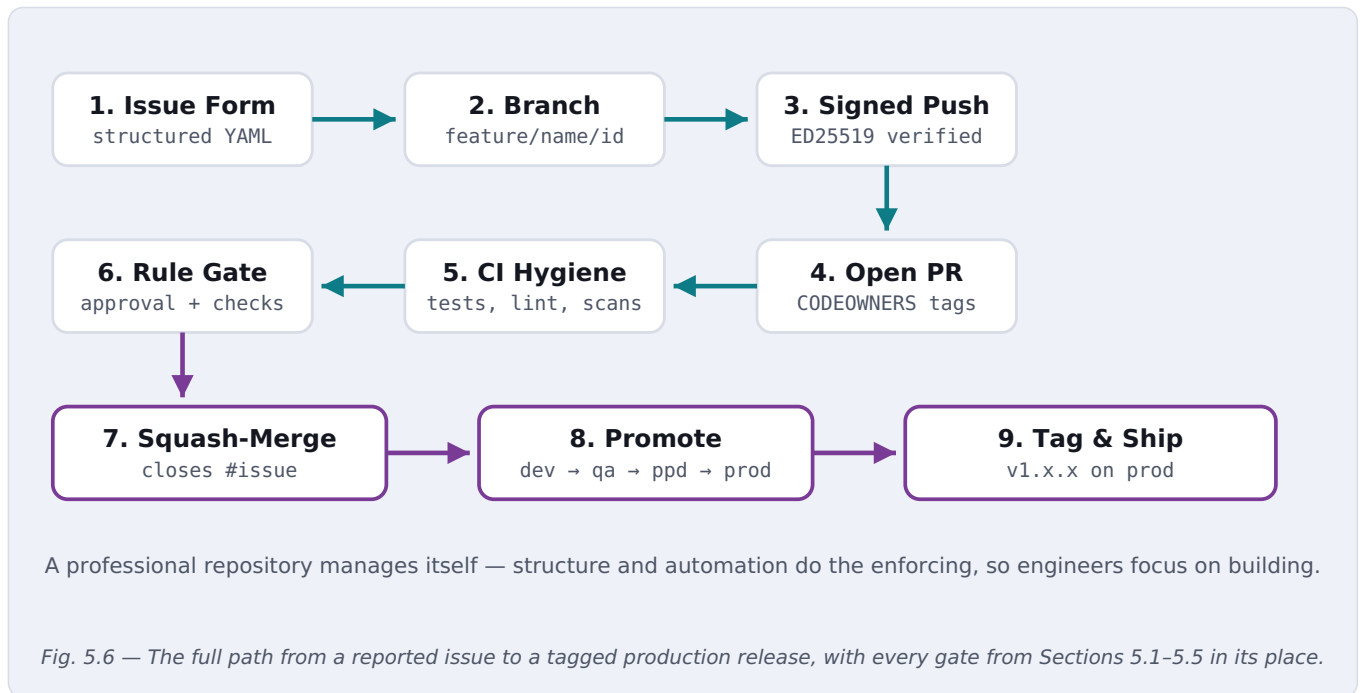
Note on scope: secret push protection only stops *new* pushes. A secret already merged into history requires the remediation in Q2/Q11 of Section 06 — rotation plus history rewrite via `git filter-repo` or BFG.

Governance Documents

Document	Audience	Purpose
README.md	Everyone	Architecture diagrams, CI status badges, local run instructions.
CONTRIBUTING.md	Developers	Branch naming, testing requirements, coding standards.
SECURITY.md	Researchers	Private vulnerability reporting — keeps zero-days off the public issue tracker.
CODE_OF_CONDUCT.md	Community	Behavioral standards and enforcement.
CHANGELOG.md	Users	Chronological record of changes per release.

5.6 The Unified Lifecycle — Everything Together

Every mechanism in this section is a stage in one continuous pipeline. This is the whole picture, end to end.



ONE-STOP TECH LEARNING PLATFORM



06 Scenario-Based Interview Questions

Twenty junior-to-mid level questions built around real situations, not command trivia.

1. You accidentally committed directly to `main` instead of a feature branch. The commit hasn't been pushed. How do you fix it?

Save the work on a new branch, then reset main: `git branch feature-fix`, `git reset --hard HEAD~1`, `git checkout feature-fix`.

Strong answer also covers — using `reset --soft HEAD~1` if you want the changes to stay staged, and confirming the rollback distance with `git log` first.

2. You pushed a commit containing a hardcoded API key to a shared branch. What do you do?

Rotate/revoke the key immediately — the exposure is real regardless of Git history. Remove it from history with `git filter-repo` or BFG Repo-Cleaner (a plain revert leaves it in prior commits), force-push the cleaned history, and have the team re-clone or hard-reset.

Strong answer also covers — pre-commit secret scanning (`gitleaks`, `git-secrets`) and GitHub's server-side push protection (Section 5.5) to prevent recurrence.

3. A teammate says "I pulled and now my branch has a weird merge commit I didn't expect." What happened, and how do you prevent it?

`git pull` defaults to fetch + merge; if their branch had diverged from the remote, Git creates a merge commit to reconcile. Prevent it with `git pull --rebase`, or `git config --global pull.rebase true`.

Strong answer also covers — the tradeoff: cleaner linear history, but never on commits already shared.

4. You need to find which of the last 200 commits introduced a bug, with no idea when it started. Walk through your approach.

`git bisect start`, mark the current commit bad and an old known-working one good. Git checks out the midpoint; test, mark, repeat — narrows to the exact commit in $O(\log n)$ steps. Finish with `git bisect reset`.

Strong answer also covers — automating the check with `git bisect run <script>`.

5. Mid-way through an interactive rebase, several commits are in the wrong state and conflicts keep appearing. What do you do?

`git rebase --abort` returns exactly to the pre-rebase state. Re-plan (perhaps touching fewer commits at once) and try again.

Strong answer also covers — `git reflog` as a safety net (Section 1.4/Section 3) — original commits stay recoverable even after a completed-but-wrong rebase, until garbage collection runs.

6. What is a "detached HEAD" state, how does someone end up there, and how do you recover?

HEAD normally points to a branch, which points to a commit (Section 1.3). Checking out a specific hash or tag directly points HEAD straight at that commit — new commits made there aren't attached to any branch and can become unreachable. Recover by branching immediately: `git checkout -b rescue-branch`.

7. `git cherry-pick` brought over more than you wanted. Why, and how do you undo it?

The source commit likely bundled the fix with unrelated changes. Undo with `git reset --hard HEAD~1` if unpushed, or `git revert <hash>` if already shared.

Strong answer also covers — `git cherry-pick -n` (no-commit) to review and selectively stage hunks first.

8. Two teammates independently created a branch named `fix/login-bug`, causing a naming collision. How should the team prevent this?

Adopt a naming convention including author or ticket ID, e.g. `fix/JIRA-123-login-bug` or the enterprise pattern `feature/{author}/{card-number}/{description}` (Section 5.3). Pair with branch protection so pushes with divergent history to an existing name are rejected.

9. A pull request shows 40 files changed, but the author only touched 2. What's going on, and how do you fix it before merging?

The branch is out of date with main and the diff includes divergent history. Rebase onto latest main, resolve conflicts, and `git push --force-with-lease` to clean up the PR diff.

Strong answer also covers — `--force-with-lease` over plain `--force` — it refuses to overwrite if someone else has pushed since your last fetch.

10. What's the difference between `git merge` and `git rebase`, and when would you choose each?

`merge` combines histories with a merge commit — non-destructive, safe on shared branches.

`rebase` replays commits onto a new base for linear history, but rewrites hashes (Section 1.1).

Guideline — rebase your own feature branch to stay current with main; merge (never rebase) when integrating a finished feature via PR.

11. A `.env` file was committed several commits ago, and is now covered by `.gitignore` — yet it still shows as tracked. Why, and how do you stop tracking it?

`.gitignore` only prevents untracked files from being added — it has no effect on files already tracked. Untrack without deleting: `git rm --cached .env`, then commit. If real secrets were pushed, also follow the rotation + history-rewrite steps from Q2.

12. You need an urgent hotfix, but your current branch has uncommitted changes you're not ready to commit. What are your options?

`git stash`, switch to main, branch for the hotfix, ship it, return and `git stash pop`. Preferred over throwaway "WIP" commits that pollute history.

Strong answer also covers — `git worktree add ../hotfix main` as an alternative that avoids stashing entirely.

13. A PR needs to combine 12 messy "WIP" / "fix typo" commits into one clean commit. How, and why does it matter?

`git rebase -i HEAD~12`, mark all but the first as `squash` or `fixup`, write one clean message — or use the host's "Squash and merge" (Section 5.5). It matters because atomic history keeps log, blame, and revert useful later.

14. Why can't Git resolve a merge conflict automatically the way it does most changes?

Git merges automatically whenever changes touch non-overlapping lines/files — a purely mechanical operation on tree/blob diffs. A conflict means the same lines changed differently on both sides, which requires understanding intent — something only a human (or explicit merge strategy) can judge. See Section 04 for the full walkthrough.

15. Your team lead asks you to set up branch protection on `main` for a new repo. What would you configure, and why?

Require PRs before merging, at least one approving review (ideally via CODEOWNERS), passing status checks, signed commits, and disallow force-pushes to main. Optionally require linear history to enforce squash/rebase-only merges. See Section 5.5 for the full ruleset.

Strong answer also covers — this directly guards against Q1 (accidental direct commits), Q9 (force-push disasters), and Q2 (unreviewed secret leaks) by making the safe path the only path.

16. A colleague asks: "if a commit is 'just a hash,' what actually gets hashed?"

The commit object's own content — tree hash, parent hash(es), author/committer info, timestamp, and message — is what gets hashed to produce the commit's ID (Section 1.1). The tree itself is a separate hash, computed from the directory listing and blob hashes it contains. Change any input and every hash downstream of it changes too, which is why editing an old commit rewrites every commit after it.

17. What's the actual difference between `git fetch` and `git pull`, mechanically?

`git fetch` downloads new objects and updates remote-tracking refs (e.g. `origin/main`) — nothing in your working branch changes. `git pull` is `fetch` immediately followed by `merge` (or `rebase`, with `--rebase`) of that remote-tracking ref into your current branch. `fetch` is always safe to run; `pull` can create a merge commit or conflict.

18. You need to undo a commit that's already been pushed and pulled by teammates. Why can't you just use `reset --hard` and force-push?

Rewriting shared history with `reset --hard` + force-push orphans everyone else's local commits built on top of the old history — their next pull either breaks or silently reintroduces the "undone" commit. Use `git revert <commit>` instead: it adds a new commit that reverses the change without altering any existing hash, so everyone's history stays consistent.

19. When would you pick Trunk-Based Development over Gitflow, and what does the team need in place first?

Trunk-Based fits high-velocity, high-maturity teams shipping continuously (Section 5.1) — but it requires two things Gitflow doesn't: near-total automated test coverage (there's no long-lived branch to catch problems before they hit trunk) and feature flags to hide incomplete work, since branch lifetimes are hours, not weeks. Without both, trunk-based development just means a constantly-broken main branch.

20. A repository has a CODEOWNERS file mapping `/terraform/*` to `@infra-team`. A PR touches both `/terraform/` and `/src/`. What happens?

GitHub reads CODEOWNERS per changed path and requests review from every matched team — here, both `@infra-team` (for the terraform changes) and whichever team owns `/src/`. If branch protection requires CODEOWNERS approval, the PR can't merge until each matched team has approved its portion, even if the other team approves quickly.

ONE-STOP TECH LEARNING PLATFORM

